

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA0605

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO1: Learning Problems, Perspectives and Issues, Concept Learning, Version Spaces & Candidate Elimination, Inductive Bias, Decision Tree Learning, Representation & Heuristic Search (BL1, BL2)

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks: 25

Annexure A

APPLICATION BASED QUESTIONS

Code of Conduct:

*I **Thamizharasan S (192424087)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Thamizharasan.s

SIMATS ENGINEERING
CO1 -ASSESSMENT TOOL 1
APPLICATION BASED
QUESTIONS

COURSE NAME: ITA0605

COURSE CODE: MACHINE LEARNING

CO1: *Learning Problems, Perspectives and Issues, Concept Learning, Version Spaces & Candidate Elimination, Inductive Bias, Decision Tree Learning, Representation & Heuristic Search (BL1, BL2)*

NAME: THAMIZHARASAN S

REG NO: 192424087

QUESTION 1:

1. In Shop X, there are 20 bottles of pure oil and 30 bottles of adulterated oil, while in Shop Y, there are 40 bottles of pure oil and 20 bottles of adulterated oil. A bottle is selected at random from one of the shops, with Shop Y being twice as likely to be chosen as Shop X. The selected bottle is found to be adulterated.

PYTHON CODE:

```
def posterior_probability():  
  
    shops = {  
        "X": {"pure": 20, "adulterated": 30, "prior": 1/3},  
        "Y": {"pure": 40, "adulterated": 20, "prior": 2/3}  
    }  
  
    likelihood = {}  
    total_probability = 0
```

```

for shop, info in shops.items():
    total = info["pure"] + info["adulterated"]
    likelihood[shop] = info["adulterated"] / total
    total_probability += likelihood[shop] * info["prior"]

print("Posterior Probabilities")
print("-" * 30)

for shop, info in shops.items():
    posterior = (likelihood[shop] * info["prior"]) / total_probability
    print(f"Shop {shop}: {posterior:.6f}")

```

posterior_probability()

OUTPUT:

```

Posterior Probabilities
-----
Shop X: 0.473684
Shop Y: 0.526316

```

QUESTION 2:

A warehouse has two sections. Section A contains 15 defective items and 35 non-defective items, while Section B contains 25 defective items and 25 non-defective items. One section is chosen at random, but Section A is three times as likely to be chosen as Section B. An item selected from the chosen section is found to be defective.

PYTHON CODE:

```

def warehouse_bayes():
    # Section data
    defective = {"A": 15, "B": 25}
    non_defective = {"A": 35, "B": 25}

    prior = {"A": 3/4, "B": 1/4}

    likelihood = {}
    total_defective_probability = 0

    for section in defective:
        total_items = defective[section] + non_defective[section]
        likelihood[section] = defective[section] / total_items

```

```

total_defective_probability += likelihood[section] * prior[section]

posterior_A = (likelihood["A"] * prior["A"]) / total_defective_probability
posterior_B = (likelihood["B"] * prior["B"]) / total_defective_probability

print("==== Bayes Theorem Result =====")
print(f'Probability of Defective Item = {total_defective_probability:.6f}')
print(f'P(Section A | Defective) = {posterior_A:.6f}')
print(f'P(Section B | Defective) = {posterior_B:.6f}')

warehouse_bayes()

```

OUTPUT:

```

==== Bayes Theorem Result =====
Probability of Defective Item = 0.350000
P(Section A | Defective) = 0.642857
P(Section B | Defective) = 0.357143

```

QUESTION 3:

In a binary classification problem using a deep learning model with a K-Nearest Neighbor classifier, the model produces True Positive (TP) = 120 and True Negative (TN) = 150. The dataset contains 1000 samples per class, and 20% of the data is used for testing.

PYTHON CODE:

```

def evaluate_model(tp, tn, total_per_class, test_ratio):

    total_test = int((2 * total_per_class) * test_ratio)
    positive_test = total_test // 2
    negative_test = total_test // 2

    fn = positive_test - tp
    fp = negative_test - tn

    accuracy = (tp + tn) / total_test
    precision = tp / (tp + fp)
    recall = tp / (tp + fn)
    specificity = tn / (tn + fp)

    print("Classification Metrics")
    print("-" * 30)

```

```

print(f"TP = {tp}, TN = {tn}, FP = {fp}, FN = {fn}")
print(f"Accuracy   : {accuracy:.4f} ({accuracy*100:.2f}%)")
print(f"Precision   : {precision:.4f} ({precision*100:.2f}%)")
print(f"Recall      : {recall:.4f} ({recall*100:.2f}%)")
print(f"Specificity : {specificity:.4f} ({specificity*100:.2f}%)")

evaluate_model(tp=120, tn=150, total_per_class=1000, test_ratio=0.20)

```

OUTPUT:

```

Classification Metrics
-----
TP = 120, TN = 150, FP = 50, FN = 80
Accuracy   : 0.6750 (67.50%)
Precision   : 0.7059 (70.59%)
Recall      : 0.6000 (60.00%)
Specificity : 0.7500 (75.00%)

```

QUESTION 4:

A medical image classification system using a deep learning framework combined with K-Nearest Neighbor (KNN) yields TP = 180 and TN = 160. Each class contains 1200 samples, and 25% of the dataset is used as test data.

PYTHON CODE:

```

class MedicalImageMetrics:
    def __init__(self, tp, tn, samples_per_class, test_ratio):
        self.tp = tp
        self.tn = tn
        self.samples_per_class = samples_per_class
        self.test_ratio = test_ratio

    def calculate_metrics(self):
        total_samples = self.samples_per_class * 2
        test_samples = int(total_samples * self.test_ratio)

        positive = test_samples // 2
        negative = test_samples // 2

        fn = positive - self.tp
        fp = negative - self.tn

        accuracy = (self.tp + self.tn) / test_samples
        precision = self.tp / (self.tp + fp)
        sensitivity = self.tp / (self.tp + fn)
        specificity = self.tn / (self.tn + fp)

```

```

print("==== Medical Image Classification Metrics =====")
print(f"TP : {self.tp}")
print(f"TN : {self.tn}")
print(f"FP : {fp}")
print(f"FN : {fn}")
print("-----")
print(f"Accuracy   : {accuracy:.4f} ({accuracy*100:.2f}%)")
print(f"Precision   : {precision:.4f} ({precision*100:.2f}%)")
print(f"Sensitivity : {sensitivity:.4f} ({sensitivity*100:.2f}%)")
print(f"Specificity : {specificity:.4f} ({specificity*100:.2f}%)")

```

```

model = MedicalImageMetrics(
    tp=180,
    tn=160,
    samples_per_class=1200,
    test_ratio=0.25
)

```

```
model.calculate_metrics()
```

OUTPUT:

```

==== Medical Image Classification Metrics =====
TP   : 180
TN   : 160
FP   : 140
FN   : 120
-----
Accuracy   : 0.5667 (56.67%)
Precision   : 0.5625 (56.25%)
Sensitivity : 0.6000 (60.00%)
Specificity : 0.5333 (53.33%)

```

QUESTION 5:

A deep learning model is trained for a classification task, and the following loss values are observed:

- i. Epoch 1: Training Loss = 2.8, Validation Loss = 2.6
- ii. Epoch 2: Training Loss = 2.4, Validation Loss = 2.2
- iii. Epoch 3: Training Loss = 2.1, Validation Loss = 1.9
- iv. Epoch 4: Training Loss = 1.9, Validation Loss = 1.8
- v. Epoch 5: Training Loss = 1.7, Validation Loss = 1.9
- vi. Epoch 6: Training Loss = 1.5, Validation Loss = 2.1
- vii. Epoch 7: Training Loss = 1.3, Validation Loss = 2.4
- viii. Epoch 8: Training Loss = 1.2, Validation Loss = 2.7

PYTHON CODE:

```
import matplotlib.pyplot as plt

epochs = list(range(1, 9))
training_loss = [2.8, 2.4, 2.1, 1.9, 1.7, 1.5, 1.3, 1.2]
validation_loss = [2.6, 2.2, 1.9, 1.8, 1.9, 2.1, 2.4, 2.7]

best_epoch = validation_loss.index(min(validation_loss)) + 1

print("Training Loss :", training_loss)
print("Validation Loss:", validation_loss)
print(f"\nBest Epoch = {best_epoch}")
print(f"Minimum Validation Loss = {min(validation_loss)}")

if validation_loss[best_epoch] > validation_loss[best_epoch - 1]:
    print("Overfitting begins after Epoch", best_epoch)

plt.figure(figsize=(7, 4))
plt.plot(epochs, training_loss, marker='o', label='Training Loss')
plt.plot(epochs, validation_loss, marker='s', label='Validation Loss')
plt.axvline(best_epoch, color='red', linestyle='--', label='Best Epoch')
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT:

Training Loss : [2.8, 2.4, 2.1, 1.9, 1.7, 1.5, 1.3, 1.2]
Validation Loss: [2.6, 2.2, 1.9, 1.8, 1.9, 2.1, 2.4, 2.7]

Best Epoch = 4

Minimum Validation Loss = 1.8

Overfitting begins after Epoch 4

